End-to-End Quality Strategy & Test Framework for Embedded Thermal Control System

Author	Date	Version	Description
Omid Madani	2026-04-02	1.0	First draft

Table of Contents

1. Scope....4
2. System Overview 2.1 System Understanding....5
2.2 QA Goal: 5
2.3 Functional Architecture....5
3. Firmware Control Loop Description 7
4. Expected Behaviors 8
4.0 Initialization Behavior 8
4.1 Temperature Control Behavior....8
4.2 PWM Behavior....8
4.3 RPM Validation Behavior 8
4.4 Fault Detection and Protection Behavior 8
4.5 Stability Requirements....9
5. Operating Modes....10
5.1 Initialization Mode....10
5.2 Normal Operation Mode 10
5.3 Validation Mode 10
5.4 Warning Mode....10
5.5 Fault Mode....10
5.6 Safe Mode....10
6. Inputs and Outputs 12
6.1 Inputs 12
6.2 Internal Signals....12
6.3 Outputs....12
7. System Requirements (High-Level) 13
8. Test Strategy....14
8.1 Validation Goals 14
8.2 Test Levels....14
8.3 Test Categories 15
8.4 Robustness Testing....16
9. Validation Methods 17
9.1 Firmware-Level Validation 17

9.2 Simulation and Fault Injection 17
9.3 Optional Hardware-Level Validation 17
10. Test Case Matrix 19
11. Pass/Fail Criteria 20
11.1 Acceptance Criteria 20
11.2 Pass/Fail Determination 20
12. Risks and Corner Cases 21
12.1 Identified Risks 21
12.2 Mitigation Strategies 21
13. CI/CD Pipeline Proposal 22
13.1 Software-Only CI Checks 22
13.2 Hardware-in-the-Loop (HIL) Validation 22
14. Traceability Matrix 23
15. Firmware Test Code Sample 24
16. Notes and Assumptions 25
17. Test Prioritization (Risk-Based) 26
18. Automation Strategy 27
19. Entry and Exit Criteria 28

1. Scope

This document defines the quality and validation strategy for a firmware-based thermal control system.

The system implements a closed-loop control mechanism in which firmware adjusts fan speed using PWM based on temperature measurements to maintain the system near a target temperature (setpoint). The focus is on verifying correct firmware behavior, validating the end-to-end closed-loop control performance, and ensuring robust operation under both nominal and fault conditions. This document focuses on system-level validation rather than low-level implementation details.

Scope

The scope of this document includes:

- Description of the temperature-to-PWM control logic
- Verification of temperature regulation performance relative to the target setpoint
- Validation of hardware–software integration across sensor, control logic, and fan actuation
- Validation of the actuation and feedback path (PWM-to-RPM relationship) using tacho measurements
- Definition of expected system behavior under normal and fault conditions
- Definition of test strategy and validation approach
- Fault scenarios and edge-case handling
- Definition of pass/fail criteria

Out of Scope

The following items are not covered in this document:

- Detailed hardware design
- Low-level driver implementation
- Advanced control algorithm design (e.g., full PID tuning)
- Manufacturing test

2. System Overview

2.1 System Understanding

This subsystem implements a closed-loop thermal control system. The firmware reads temperature data via I2C, compares it to a target setpoint, and computes the temperature error. Based on this error, the firmware adjusts the fan drive using a defined PWM mapping. The resulting fan response is measured through tachometer (tacho) feedback, allowing the system to validate that the achieved RPM aligns with the expected behavior for the applied PWM.

From a QA perspective, the objective is not only to verify individual interfaces such as I2C communication, PWM generation, or tacho measurement in isolation, but to demonstrate that the complete control loop operates correctly as an integrated system. This includes ensuring that temperature changes lead to appropriate control actions and that the physical response of the fan is consistent with system expectations.

The system consists of:

- Microcontroller (MCU) running firmware
- Temperature sensor (I2C-based)
- PWM-controlled fan
- Tacho feedback signal

2.2 QA Goal:

Verify that the firmware maintains the measured temperature close to a defined target setpoint by adjusting fan speed, while ensuring safe and robust operation under both nominal and fault conditions. The validation approach treats the firmware as a black box, focusing on verifying that physical outputs (PWM duty cycle and resulting fan speed) correctly respond to system inputs such as temperature and control commands.

The validation focuses on the actuation and feedback path, ensuring that commanded PWM signals produce the expected physical fan response (RPM) within defined tolerances.

2.3 Functional Architecture

The system is divided into two logical paths (see Figure 1):

Control Path

- Temperature is measured
- Compared with target temperature
- PWM duty cycle is adjusted according to the error (measured temperature – target temperature)

Validation Path

- PWM is applied to fan
- Fan generates RPM
- Tacho signal provides feedback
- Firmware verifies that RPM matches the expected value

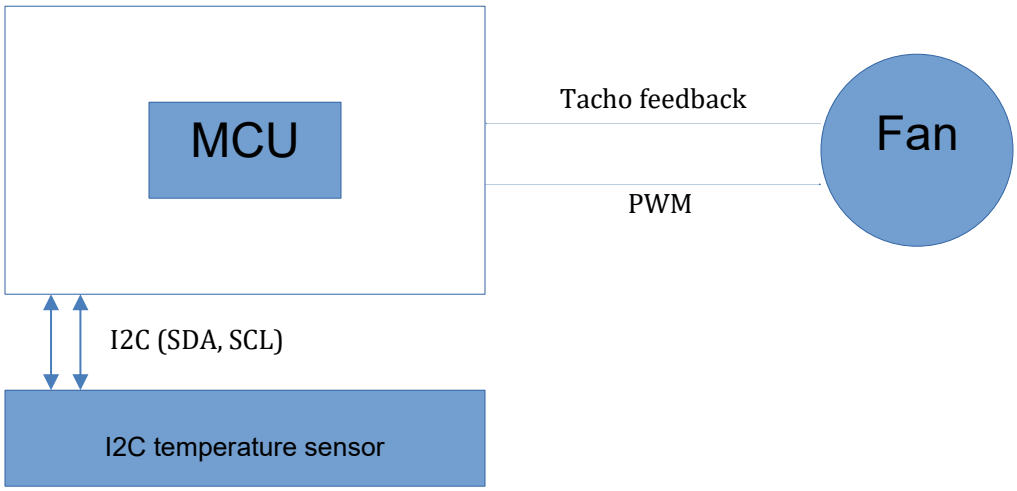

Figure 1 The system consists of an MCU controlling a PWM-driven fan and an I2C temperature sensor. The fan provides tacho feedback, and the firmware dynamically adjusts the fan speed to maintain the target temperature.

3. Firmware Control Loop Description

The firmware implements a simple closed-loop temperature control with validation feedback.

Key Concept

- Control is based on temperature → PWM
- Validation ensures PWM produces correct RPM

Control Flow

1. Read temperature via I2C
2. Compute temperature error as: error = measured temperature - target temperature
3. Convert error to PWM using a lookup table
4. Apply PWM signal to fan

Validation Flow

5. Measure tacho pulses and calculate RPM
6. Determine expected RPM based on PWM using a lookup table
7. Compare measured RPM with expected RPM
8. If deviation exceeds the threshold, flag a warning or fault

4. Expected Behaviors

4.0 Initialization Behavior

During system startup, I2C communication with the temperature sensor and PWM output configuration shall be successfully initialized and verified before entering normal operation.

4.1 Temperature Control Behavior

Nominal Operation (Temperature Above Target)

- When the measured temperature rises above the target, the commanded PWM duty cycle shall increase accordingly.
- The fan speed (RPM) shall follow the expected value within an allowed settling time based on the PWM–RPM mapping.

Recovery Behavior (Cooling Toward Target)

- As the temperature decreases toward the target, the commanded PWM duty cycle shall decrease accordingly.
- PWM reduction shall be smooth and stable, avoiding excessive oscillation or hunting.
- The system shall converge toward the target temperature within a defined tolerance band.

4.2 PWM Behavior

- PWM duty cycle shall follow the defined control strategy (lookup table or proportional mapping).
- PWM updates shall be smooth and bounded, avoiding abrupt or unstable transitions unless required for protection.
- PWM values shall remain within valid limits (e.g., 0% to 100%).

4.3 RPM Validation Behavior

- The measured fan speed (RPM), derived from the tacho signal, shall match the expected RPM for a given PWM duty cycle within a defined tolerance (e.g., ±10%).
- The system shall detect deviations between expected and measured RPM values.
- RPM shall reach the expected range within a defined settling time after a PWM change.

4.4 Fault Detection and Protection Behavior

Sensor-Related Faults

- If temperature sensor data is invalid, missing, or out of range:
  - The system shall enter a safe state.
  - A fault shall be raised.
  - A fallback control strategy shall be applied (e.g., fixed safe PWM such as 100% duty cycle) to ensure adequate cooling and prevent hardware damage.

Fan / Tacho Faults

- If no tacho signal is detected within a defined timeout:
  - The system shall raise a fan fault.
- If measured RPM is significantly lower than expected for a given PWM:
  - The system shall detect a performance degradation or fan failure condition.

4.5 Stability Requirements

- The control response shall be stable under normal operating conditions.
- The system shall avoid oscillatory behavior, rapid PWM toggling, or unstable control loops.
- Small temperature variations near the target shall not cause excessive PWM changes.

5. Operating Modes

The system operates in several defined modes to ensure correct functionality, monitoring, and safe behavior under all conditions.

5.1 Initialization Mode

- During startup, the system shall initialize I2C communication with the temperature sensor and configure the PWM output.
- The system shall verify that all required interfaces are operational before transitioning to normal operation.

5.2 Normal Operation Mode

- The system shall continuously read temperature data and adjust PWM duty cycle based on the control strategy (temperature-to-PWM mapping).
- The fan speed shall respond accordingly to maintain the temperature near the target setpoint.

5.3 Validation Mode

- The system shall continuously monitor tacho feedback and calculate fan speed (RPM).
- The measured RPM shall be compared against the expected RPM based on PWM.
- Deviations beyond defined thresholds shall trigger warning or fault conditions.

5.4 Warning Mode

- The system shall enter warning mode when deviations between expected and measured RPM exceed predefined thresholds but the system remains operational.
- Warning conditions shall be logged for further analysis.

5.5 Fault Mode

- The system shall enter fault mode when critical failures are detected, such as:
    - Missing tacho signal
    - Fan stall condition
    - Invalid or missing sensor data
- The system shall raise appropriate fault flags and initiate protective actions.

5.6 Safe Mode

- In fault conditions, the system shall transition to a safe mode to prevent hardware damage.

- A fallback control strategy shall be applied (e.g., fixed high PWM such as 100% duty cycle) to ensure sufficient cooling.

6. Inputs and Outputs

The system processes input signals, computes internal control and validation parameters, and generates outputs to control and monitor system behavior.

6.1 Inputs

- Temperature (I2C):
  The measured temperature value is read from the I2C sensor and used as the primary input for control decisions.
- Tacho signal (digital input):
  Pulse signals from the fan are used to calculate the measured fan speed (RPM).

6.2 Internal Signals

- Temperature error:
  The difference between measured temperature and target temperature, used to determine the required PWM adjustment.
- Expected PWM:
  The PWM duty cycle derived from the temperature error using the defined control mapping.
- Expected RPM:
  The target fan speed corresponding to the commanded PWM duty cycle based on the PWM–RPM mapping.
- Calculated RPM:
  The measured fan speed computed from tacho pulse measurements.

6.3 Outputs

- PWM signal:
  The control signal applied to the fan to adjust its speed.
- Fault flags:
  Indicators raised when abnormal conditions or failures are detected (e.g., missing tacho, sensor error).
- Debug and log output:
  Logged data including temperature, PWM, RPM, and system status for validation and diagnostics.

7. System Requirements (High-Level)

The following high-level requirements define the expected system behavior and provide measurable criteria for validation.

ID	Requirement	Acceptance / Measurable Criteria (example - adjust)
R1	Maintain target temperature	Steady-state temperature remains within a defined tolerance band around the target (e.g., ±1°C) after settling time
R2	Adjust fan speed based on temperature	PWM duty cycle increases or decreases consistently with temperature error; no stuck or non-responsive behavior
R3	Accurate RPM feedback	Measured RPM remains within defined tolerance (e.g., ±10%) of expected RPM based on PWM–RPM mapping
R4	Detect fan faults	Fan stall or tacho mismatch is detected within a defined timeout and appropriate fault flags are raised
R5	Handle sensor failures safely	I2C read failures, invalid data, or out-of-range values are detected within a defined time (e.g., ≤1 second), and the system enters safe mode with appropriate fault indication
R6	Maintain stable operation	The system maintains stable operation near the target temperature without oscillation, chattering, or rapid PWM fluctuations beyond defined limits

Table 1 High-Level System Requirements and Acceptance Criteria

8. Test Strategy

The validation strategy focuses on functional correctness, system stability, and hardware–software correlation across all test levels.

8.1 Validation Goals

- **Correctness:**
  Output behavior matches the intended thermal control response.
- **Stability:**
  No uncontrolled oscillation, chattering, or unstable response near thresholds.
- **Observability:**
  Logs, counters, and status signals allow failures to be diagnosed.
- **Safety:**
  Invalid inputs and hardware faults are detected and handled gracefully.
- **Repeatability:**
  Tests can be executed automatically on bench hardware as part of regression.

8.2 Test Levels

Validation is performed across multiple test levels to ensure correct behavior at unit, integration, and system levels.

- **Unit Testing (where applicable):**
  Verification of individual firmware components such as:
  - Temperature sensor data parsing and error handling
  - PWM mapping and saturation logic
  - Tacho pulse-to-RPM conversion (including edge cases such as timer overflow)
  - Fault detection and state machine transitions
- **Interface-Level Checks:**
  Verification of individual interfaces in isolation, including sensor communication (I2C), PWM signal generation, and tacho pulse measurement.
- **Firmware Functional Checks:**
  Verification of control decisions, thresholds, duty limits, fault flags, and startup/shutdown behavior.
- **Hardware–Software Integration Checks:**
  Verification that commanded PWM signals result in corresponding physical fan behavior (RPM), including correct tacho feedback processing under real conditions.

- **System-Level Testing:**
  End-to-end validation of the closed-loop thermal control behavior, including stability, response to temperature changes, and safe-mode operation under fault conditions.
- **Hardware-in-the-Loop (HIL) / Test Rig:**
  Automated test execution using scripts interacting with real hardware, including PWM control, tacho measurement, and fault injection. This setup enables repeatable regression testing and can be integrated into CI workflows where feasible.

8.3 Test Categories

- **Functional Testing:**
  Verify temperature-to-PWM mapping and correct response to temperature changes.
- **Boundary and Limit Testing:**
  Verify behavior at minimum and maximum temperature and PWM limits, including saturation and recovery behavior.
- **Negative and Robustness Testing:**
  Verify correct handling of abnormal or invalid conditions such as noisy sensor readings, stuck-at values, or out-of-range inputs.
- **Fault Injection Testing:**
  Validate system behavior under fault conditions, including missing tacho signal, invalid temperature data, fan underperformance, and sensor communication failures.
- **Performance Testing:**
  Evaluate response time, settling time, and timing consistency of the control and validation paths.
- **Stability and Control-Quality Testing:**
  Verify that the control loop remains stable near the target temperature without excessive oscillation, overshoot, or hunting.
- **Stress Testing:**
  Verify system behavior under rapid temperature changes, repeated PWM updates, and repeated fault/recovery cycling.
- **Reliability / Soak Testing:**
  Run the system over an extended period to detect intermittent failures, drift, or degradation in control and monitoring behavior.
- **Regression Testing:**
  Execute automated tests to ensure stability and prevent regressions after firmware changes.
- **Observability and Diagnostics Testing:**
  Verify that logs, counters, status indicators, and fault codes provide sufficient visibility to diagnose failures and validate system behavior.

-

8.4 Robustness Testing

- **System robustness checks:**
  Verify behavior under abnormal and edge conditions, including:
    - Noisy sensor readings
    - Missing tacho pulses
    - Stalled fan conditions
    - Stuck-at values
    - Out-of-range temperature inputs

9. Validation Methods

Validation is performed using a combination of firmware-level measurements and optional hardware-level observation tools to verify system behavior and ensure consistency between commanded control signals and physical response.

9.1 Firmware-Level Validation

- **Tacho-based RPM measurement:**
  Fan speed (RPM) is calculated from tacho pulse signals and used to validate PWM-to-RPM behavior.
- **Logging and telemetry:**
  The system logs temperature, commanded PWM, measured RPM, fault flags, and timestamps to enable analysis and debugging.
  This data is used to evaluate system stability, response time, and steady-state performance.
- **Script-based validation:**
  Automated scripts (Python or C) are used to compare expected and measured behavior, including tolerance checks and pass/fail evaluation.

9.2 Simulation and Fault Injection

- **Simulated I2C inputs:**
  Temperature input can be simulated by mocking the I2C sensor interface or by using an I2C emulator. This allows controlled injection of temperature profiles to verify firmware response in terms of PWM and RPM.
- **Deterministic and repeatable testing:**
  This approach enables deterministic and repeatable testing, which is essential for automated validation and regression.
- **Fault injection via script:**
  Fault scenarios such as missing tacho signal, invalid sensor data, or out-of-range values are simulated to verify system robustness.
- **Temperature override (if supported):**
  If supported by the firmware, a temperature override register can be used to inject controlled values without modifying firmware behavior. If such a mechanism is not available, an external I2C sensor emulator can be used to achieve the same effect.

9.3 Optional Hardware-Level Validation

- **Oscilloscope:**
  Used to verify PWM signal characteristics such as duty cycle and frequency.

- **Logic analyzer:**
  Used to inspect signal timing and communication interfaces (e.g., I2C, tacho pulses).
- **Controlled thermal setup (optional):**
  For full closed-loop validation, a controlled thermal setup (e.g., heater or thermal chamber) can be used to stimulate the temperature sensor and observe system response over time.

10. Test Case Matrix

The following test cases represent a selected set of high-priority and representative scenarios used to validate the most critical functional, safety, stability, and fault-handling behaviors of the subsystem. Additional test cases may be defined during detailed validation planning and execution.

ID	Scenario	Stimulus / Method	Expected Result	Priority
TC-01	PWM-to-RPM mapping	Apply multiple PWM values	RPM follows expected curve within tolerance	P0
TC-02	Startup behavior	Enable fan from zero	Fan starts within timeout	P0
TC-03	Fan stall	Block fan or remove tacho	Fault detected and safe mode activated	P0
TC-04	Sensor fault	Inject I2C error	Fault detected and fallback applied	P0
TC-05	Stability near setpoint	Sweep temperature around target	No oscillation or hunting	P0
TC-06	Boundary conditions	Apply extreme temperature values	System clamps and behaves safely	P1
TC-07	Missing tacho	Disconnect tacho signal	Fault detected within timeout	P0
TC-08	Low RPM vs PWM	Inject reduced RPM	Warning or fault triggered	P1
TC-09	Fault recovery	Inject and clear fault	System returns to normal operation	P1

Table 2 Representative Test Cases for Functional, Fault, and Stability Validation

11. Pass/Fail Criteria

11.1 Acceptance Criteria

The system shall meet the following criteria during validation:

- PWM duty cycle shall remain within defined minimum and maximum bounds.
- Measured RPM shall track the commanded PWM within a defined tolerance band (e.g., ±10%) after a specified settling time.
- Loss of tacho signal or fan stall shall be detected within a bounded timeout.
- I2C read failures, CRC issues (if applicable), or out-of-range sensor values shall trigger deterministic fault handling.
- Logs shall capture the following:
    - Timestamp
    - Commanded PWM
    - Measured RPM
    - Temperature
    - Pass/fail result
    - Active fault flags

11.2 Pass/Fail Determination

A test is considered PASS if all acceptance criteria are satisfied.

A test shall be considered FAIL if any of the following conditions occur:

- PWM does not respond correctly to temperature changes.
- Measured RPM deviates beyond the defined tolerance range.
- Fault detection or handling does not behave as specified.

12. Risks and Corner Cases

12.1 Identified Risks

The system may be affected by the following key risks and critical areas:

- **Safety risk:**
  Overheating may occur if the fan fails or if temperature sensor data becomes invalid or unavailable.
- **Stability risk:**
  Control loop oscillations may lead to excessive noise, increased mechanical wear, and poor thermal regulation.
- **Measurement accuracy risk:**
  Tacho signal misinterpretation due to noise, incorrect pulses-per-revolution (PPR) configuration, or timer overflow may result in inaccurate RPM calculation.
- **I2C communication robustness risk:**
  Bus hangs, intermittent NACK responses, or corrupted sensor data may affect system reliability.
- **Timing risk:**
  Control loop jitter or inconsistent update timing may degrade control performance and lead to instability.
- **Hardware variability risk:**
  Variations in fan characteristics (aging, non-linearity) may impact PWM-to-RPM mapping accuracy.

12.2 Mitigation Strategies

The following mitigation mechanisms shall be applied:

- **Tolerance thresholds:**
  Defined tolerance bands shall be used to account for expected variations in RPM and temperature.
- **Filtering:**
  Sensor readings shall be filtered to reduce the impact of noise and transient fluctuations.
- **Fault detection mechanisms:**
  The system shall detect abnormal conditions (e.g., missing tacho, sensor failure) and trigger appropriate fault handling.

13. CI/CD Pipeline Proposal

A practical CI/CD approach should support both software-only validation and hardware-assisted validation where applicable.

13.1 Software-Only CI Checks

For environments without direct access to hardware, the following checks shall be executed automatically:

- Build the firmware image.
- Run static analysis on the firmware codebase.
- Execute host-based unit tests for key logic such as PWM mapping, tacho-to-RPM conversion, and fault handling.
- Run simulation-based tests using golden vectors to validate control calculations and RPM conversion behavior.

13.2 Hardware-in-the-Loop (HIL) Validation

Where a hardware test rig is available, a nightly or gated HIL job can be executed as follows:

- Flash firmware to the bench target.
- Run a hardware smoke test to confirm communication and firmware version.
- Execute a PWM sweep and verify measured RPM against defined tolerance limits.
- Execute sensor fault injection and verify safe-mode behavior.
- Archive logs, verdict artifacts, and plots for traceability and debugging.
- Fail the pipeline if any measured point is outside tolerance or if communication fails.

14. Traceability Matrix

This table includes a representative subset of test scenarios. A complete validation plan would include additional detailed test cases.

Requirement ID	Requirement Description	Related Test Cases	Coverage Intent
R1	Maintain target temperature within defined tolerance after settling	TC-05, TC-06	Functional thermal control, stability near setpoint, boundary behavior
R2	Adjust fan speed based on temperature error	TC-01, TC-05	Temperature-to-PWM control behavior, commanded response
R3	Ensure accurate RPM feedback and PWM-to-RPM correlation	TC-01, TC-02, TC-08	RPM validation, startup response, low-RPM mismatch detection
R4	Detect fan faults and take safe action	TC-03, TC-07, TC-08	Fan stall detection, missing tacho detection, degraded fan response
R5	Handle sensor failures safely	TC-04, TC-06	Sensor communication fault handling, invalid/out-of-range input handling
R6	Maintain stable operation without excessive oscillation or hunting	TC-05, TC-09	Stability, recovery behavior, controlled return to normal operation

Table 3 Traceability Matrix Between High-Level Requirements and Representative Test Cases

15. Firmware Test Code Sample

This script implements a test framework to validate the fan actuation and feedback path by comparing commanded PWM duty cycle with measured fan speed (RPM via tacho).

The script can operate either with a real device under test (via serial CLI) or in a simulated mode for development and fault injection.

The test framework performs automated pass/fail evaluation based on defined tolerance criteria and supports logging for further analysis and traceability.

Supported fault simulation modes include the following:

- stuck_fan
- no_tacho
- tacho_low

16. Notes and Assumptions

The following assumptions are made for the validation approach:

- The device under test (DUT) provides a serial interface or command-line interface (CLI) for setting PWM duty cycle and reading measured RPM values.
- If no CLI is available, the same validation approach can be adapted using alternative interfaces such as debug registers, SWD commands, CAN, USB, or a custom lab control harness.
- Tolerance values and timing limits (e.g., RPM tolerance, settling time, fault detection timeout) shall be defined in collaboration with design owners based on fan datasheet specifications, tacho pulse-per-revolution characteristics, and mechanical system behavior.

Additional validation considerations:

- This script validates the PWM-to-tacho path (open-loop behavior only). Full closed-loop thermal validation, including temperature-driven control dynamics, is covered at system level.
- The expected RPM values are representative for this exercise. In a real system, they would be derived from requirements, fan datasheets, and characterization data.
- A complete validation should also ensure that increasing PWM results in non-decreasing RPM, to detect mapping or control issues.
- RPM can be sampled multiple times and averaged to reduce noise and improve measurement accuracy.

17. Test Prioritization (Risk-Based)

Test execution is prioritized based on system criticality, safety impact, and likelihood of failure.

P0 – Must Pass (Release Gates)

- Core thermal control functionality (temperature-to-PWM behavior)
- Sensor and fan fault detection and safe-mode handling
- RPM validation and tacho integrity under nominal conditions
- Basic stability verification (no uncontrolled oscillation or runaway behavior)

P1 – Important

- Performance metrics such as response time and settling time
- Additional fault scenarios (e.g., noisy tacho signal, intermittent I2C communication)

P2 / P3 – Lower Priority

- Extended stress and corner-case scenarios
- Long-duration soak and reliability testing

18. Automation Strategy

Validation activities are designed to maximize automation while acknowledging practical hardware constraints.

Fully Automated

- Sensor communication validation, including valid and invalid I2C transactions
- PWM-to-RPM characterization and tolerance verification
- Fault injection through software or electrical means (e.g., simulated tacho loss, I2C errors)
- Regression test execution triggered on code changes (e.g., CI/CD pipeline)

Semi-Automated

- Physical fault scenarios such as fan stall, requiring manual setup with automated measurement and logging
- Hardware interaction tests where setup is manual but execution and validation are scripted

Manual

- Visual inspection and hardware setup verification
- Investigation of unexpected hardware behavior or anomalies

19. Entry and Exit Criteria

Entry Criteria

The following conditions shall be met before validation activities begin:

- Firmware features for the targeted release are implemented and available for testing
- Hardware setup is defined, stable, and reproducible
- Acceptance criteria for key system requirements (R1–R6) are defined, even if initial thresholds are provisional

Exit Criteria (Release Candidate)

The system shall be considered ready for release when the following conditions are satisfied:

- 100% of P0 (must-pass) test cases have passed
- No open critical or high-severity defects remain for the subsystem
- Performance and stability criteria are met, or any deviations are formally reviewed and accepted
- Regression test suite passes on the latest firmware build